

coursework_01

January 29, 2026

1 Coursework 1: Image filtering

The coursework includes coding questions and/or written questions. Please read both the text and the code in this notebook to get an idea what you are expected to implement.

1.1 What is expected?

- Complete and run the code using `jupyter-lab`.
- Export (File | Save and Export Notebook As...) the notebook as a PDF file, which contains your code, results and answers, and upload the PDF file onto [Scientia](#).
- Instead of clicking the Export button, you can also run the following command instead:
`jupyter nbconvert coursework_01.ipynb --to pdf`
- If Jupyter complains issues during exporting, it is likely that [pandoc](#) or latex is not installed, or their paths have not been included. You can install the relevant libraries and retry. Alternatively, use the Print function of your browser to export the PDF file.
- If Jupyter-lab does not work for you at the end, alternatively, you can use Google Colab to write the code and export the PDF file.

1.2 Dependencies:

You may need to install [Jupyter-Lab](#) and other libraries used in this coursework, such as by running the command: `pip3 install [package_name]`

```
[1]: # Import libraries (provided)
import imageio.v3 as imageio
import numpy as np
import matplotlib.pyplot as plt
import scipy
import scipy.signal
import math
import time
```

1.3 Q1. Moving average filtering (20 points).

Read the provided clean image, add noise to the image and design a moving average filter for denoising.

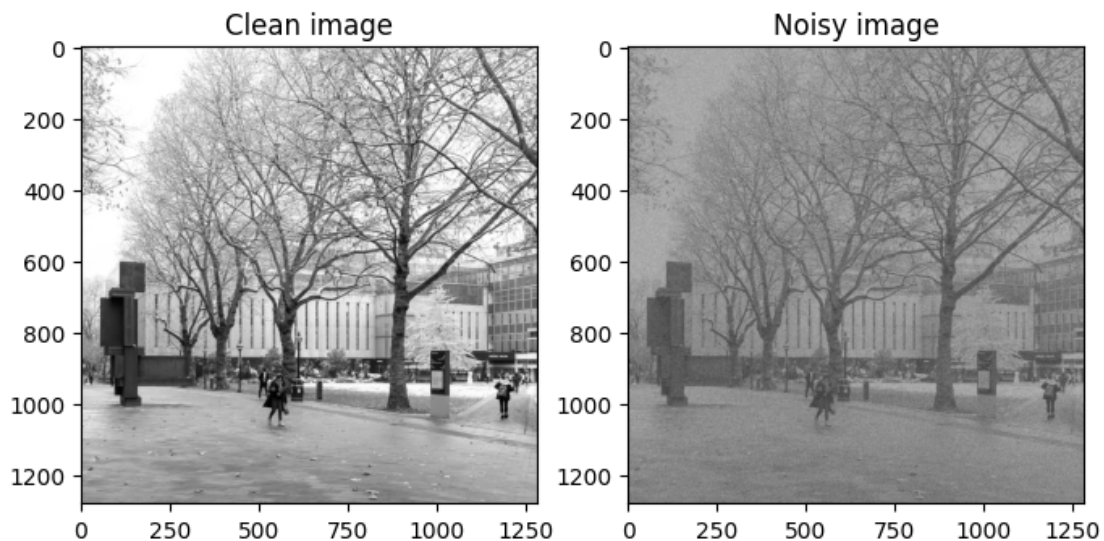
You are expected to design the kernel of the filter and then perform 2D image filtering using the function `scipy.signal.convolve2d()`.

```
[2]: # Read the image (provided)
image = imageio.imread('campus_snow.jpg')

# Corrupt the image with Gaussian noise (provided)
noise_mu = 0
noise_sigma = 50
noise = np.random.normal(noise_mu, noise_sigma, image.shape)
image_noisy = image + noise

# Visualise the images (provided)
plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Clean image')

plt.subplot(1, 2, 2)
plt.imshow(image_noisy, cmap='gray')
plt.title('Noisy image')
plt.gcf().set_size_inches(8, 4)
```



1.3.1 Q1.1 Filter the noisy image using a 5x5 moving average filter. Display the filtered image.

```
[3]: # Design the filter h
### Insert your code ###
h = np.ones((5, 5)) / 25
```

```

# Convolve the noisy image with h using scipy.signal.convolve2d function
### Insert your code ###
image_filtered = scipy.signal.convolve2d(
    image_noisy,
    h,
    mode='same',
    boundary='symm'
)

# Print the filter (provided)
print('Filter h = {0}'.format(h))

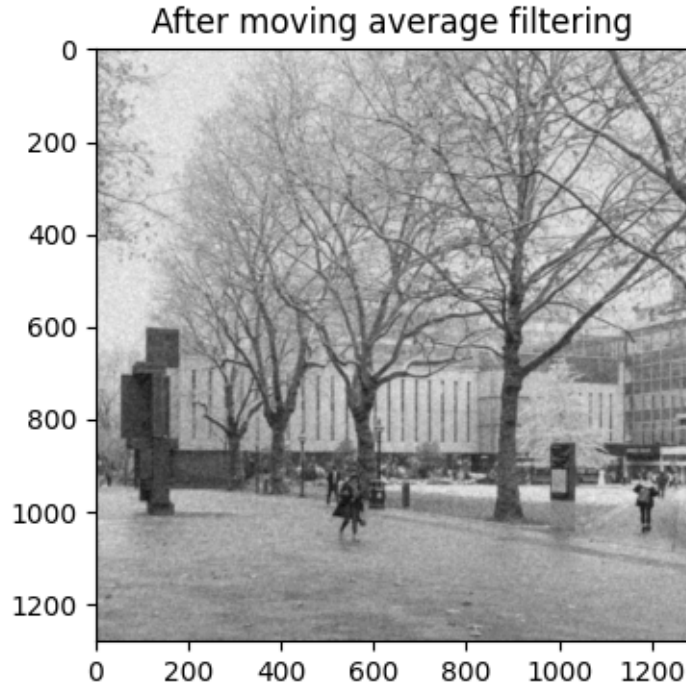
# Display the filtering result (provided)
plt.imshow(image_filtered, cmap='gray')
plt.title('After moving average filtering')
plt.gcf().set_size_inches(4, 4)

```

```

Filter h = [[0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]
 [0.04 0.04 0.04 0.04 0.04]]

```



1.3.2 Q1.2 Assess the quality of the filtered image using a quantitative metric, the peak signal-to-noise ratio (PSNR).

For this case, the pixel intensity of the image is represented using the uint8 format, with the peak value to be 255. The PSNR is defined as,

$$\text{PSNR} = 10 \cdot \log_{10} \frac{255^2}{\frac{1}{N} \sum_x [J(x) - I(x)]^2}$$

where x denotes the pixel index, N denotes the total number of pixels in the image, J denotes the filtered i.e. denoised image and I denotes the ground truth clean image. The denominator of the term within the logarithm operator is the mean squared error between I and J .

You can find more detail about PSNR [here](#).

```
[4]: # Implement the PSNR function
def eval_psnr(I, J):
    # I: the ground truth clean image (peak value: 255 for uint8 data format)
    # J: the denoised image
    #
    # return: the PSNR metric (unit: dB)

    ### Insert your code ###
    I = I.astype(np.float64)
    J = J.astype(np.float64)
    mse = np.mean((I - J) ** 2)
    if mse == 0:
        return float('inf')
    max_pixel = 255.0
    psnr = 10 * np.log10((max_pixel ** 2) / mse)
    return psnr

# Evaluate the PSNR for the filtered image (provided)
psnr = eval_psnr(image, image_filtered)

# Print the PSNR (provided)
print('PSNR = {0:.2f} dB'.format(psnr))
```

PSNR = 18.49 dB

1.4 Q2. Gaussian filtering (70 points).

1.4.1 Q2.1 Implement a function that constructs a 2D Gaussian filter given the parameter σ .

```
[5]: # Implement the Gaussian filter
def gaussian_filter_2d(sigma):
    # sigma: the parameter sigma for the Gaussian kernel (unit: pixel)
    #
    # return: a 2D array for the Gaussian kernel
```

```

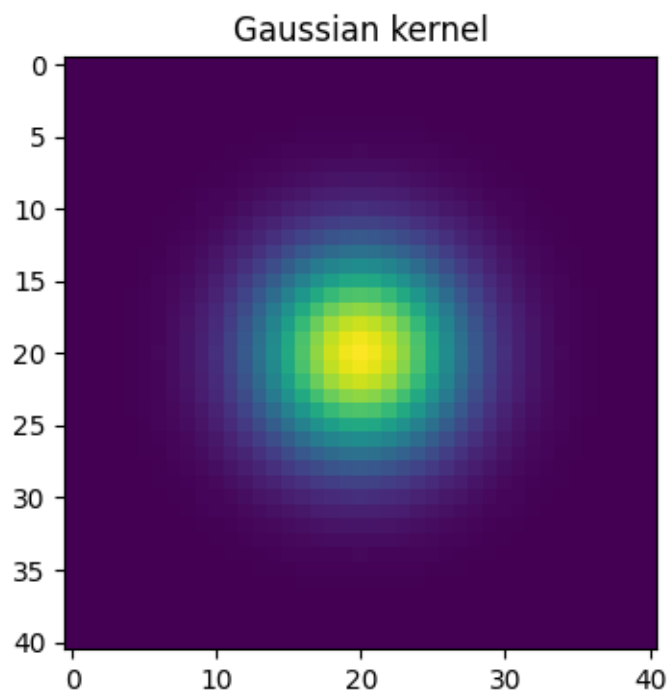
# The filter radius is 4 times sigma (provided)
rad = int(math.ceil(4 * sigma))
sz = 2 * rad + 1

# Calculate the filter weights
### Insert your code ###
ax = np.arange(-rad, rad + 1)
xx, yy = np.meshgrid(ax, ax)
h = np.exp(-(xx**2 + yy**2) / (2 * sigma**2))
h = h / np.sum(h)

return h

# Visualise the Gaussian filter when sigma = 5 pixel (provided)
sigma = 5
h = gaussian_filter_2d(sigma)
plt.imshow(h)
plt.title('Gaussian kernel')
plt.gcf().set_size_inches(4, 4)

```



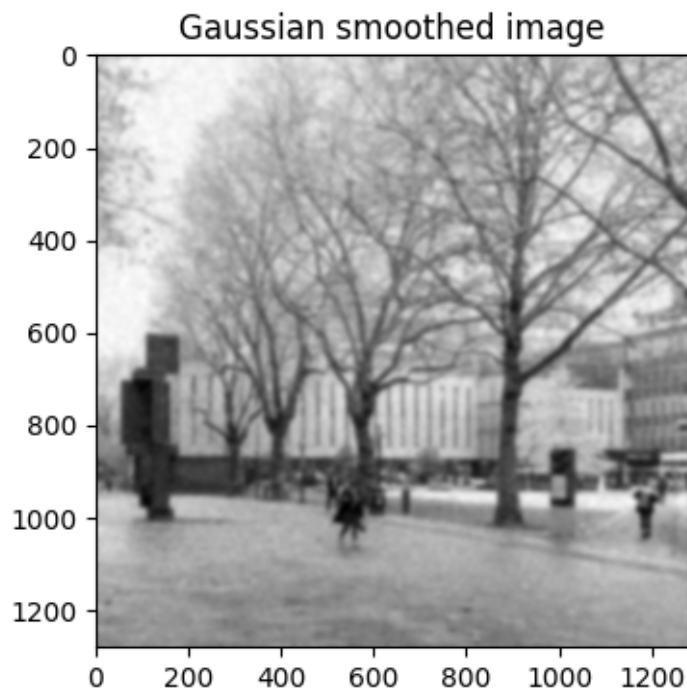
1.4.2 Q2.2 Perform Gaussian filtering ($\sigma = 5$ pixels) for the noisy image, evaluate the computational time for Gaussian filtering and display the filtered image.

```
[6]: # Construct the Gaussian filter (provided)
sigma = 5
h = gaussian_filter_2d(sigma)

# Perform Gaussian filtering and count time
### Insert your code ###
start_time = time.time()
image_smoothed_2d = scipy.signal.convolve2d(
    image_noisy,
    h,
    mode='same',
    boundary='symm'
)
t_2d = time.time() - start_time
print("2D Gaussian filtering time: {:.4f} s".format(t_2d))

# Visualise the filtered image (provided)
plt.imshow(image_smoothed_2d, cmap='gray')
plt.title('Gaussian smoothed image')
plt.gcf().set_size_inches(4, 4)
```

2D Gaussian filtering time: 3.5042 s



1.4.3 Q2.3 Implement a function that generates a 1D Gaussian filter given the parameter σ . Construct 1D Gaussian filters along x-axis and y-axis respectively.

```
[7]: # Implement the 1D Gaussian filter
def gaussian_filter_1d(sigma):
    # sigma: the parameter sigma in the Gaussian kernel (unit: pixel)
    #
    # return: a 1D array for the Gaussian kernel

    # The filter radius is 4 times sigma (provided)
    rad = int(math.ceil(4 * sigma))
    sz = 2 * rad + 1

    # Calculate the filter weights
    ### Insert your code ###
    ax = np.arange(-rad, rad + 1)
    h = np.exp(-(ax**2) / (2 * sigma**2))
    h = h / np.sum(h) # normalize
    return h

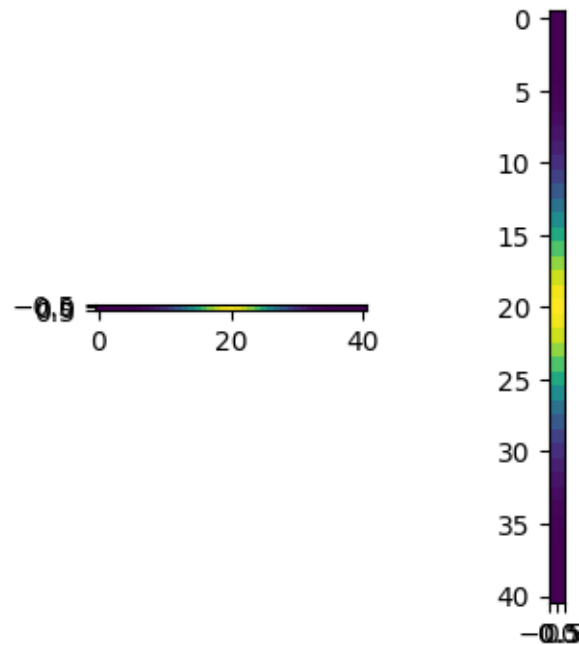
# sigma = 5 pixel (provided)
sigma = 5

# Construct the Gaussian filter along x-axis. Its shape is (1, sz).
### Insert your code ###
g = gaussian_filter_1d(sigma)
h_x = g.reshape(1, -1)

# Construct the Gaussian filter along y-axis. Its shape is (sz, 1).
### Insert your code ###
h_y = g.reshape(-1, 1)

# Visualise the filters (provided)
plt.subplot(1, 2, 1)
plt.imshow(h_x)

plt.subplot(1, 2, 2)
plt.imshow(h_y)
plt.gcf().set_size_inches(4, 4)
```



1.4.4 Q2.4 Perform Gaussian filtering ($\sigma = 5$ pixels) using two separable filters and evaluate the computational time for separable Gaussian filtering. Compare the smoothed image using separable filtering to the smoothed image using a single 2D Gaussian filter.

```
[8]: # Perform separable Gaussian smoothing and count time
    ### Insert your code ###
    start_time = time.time()
    temp = scipy.signal.convolve2d(
        image_noisy,
        h_x,
        mode='same',
        boundary='symm'
    )
    image_smoothed = scipy.signal.convolve2d(
        temp,
        h_y,
        mode='same',
        boundary='symm'
    )
    t_sep = time.time() - start_time
    print("Separable Gaussian filtering time: {:.4f} s".format(t_sep))
```



```

# Report the difference between the separably filtered image and the image
↳ filtered by a single 2D Gaussian filter (provided)
diff = image_smoothed - image_smoothed_2d
print('Mean absolute difference = {:.6f}'.format(np.mean(np.abs(diff))))

```

Separable Gaussian filtering time: 0.9640 s

Mean absolute difference = 0.000000

1.4.5 Q2.5 Perform Gaussian smoothing for the same noisy image, assess the quality of the Gaussian smoothed image using PSNR, when different sigma values are used.

```

[9]: # A list of sigma values (provided)
list_sigma = np.arange(0.5, 5, 0.5)

# Perform Gaussian smoothing with different sigma values and record the PSNR
↳ values
### Insert your code ###
list_psnr = []

for sigma in list_sigma:
    g = gaussian_filter_1d(sigma)
    h_x = g.reshape(1, -1)
    h_y = g.reshape(-1, 1)

    tmp = scipy.signal.convolve2d(
        image_noisy,
        h_x,
        mode='same',
        boundary='symm'
    )

    image_smoothed = scipy.signal.convolve2d(
        tmp,
        h_y,
        mode='same',
        boundary='symm'
    )

    psnr = eval_psnr(image, image_smoothed)
    list_psnr.append(psnr)

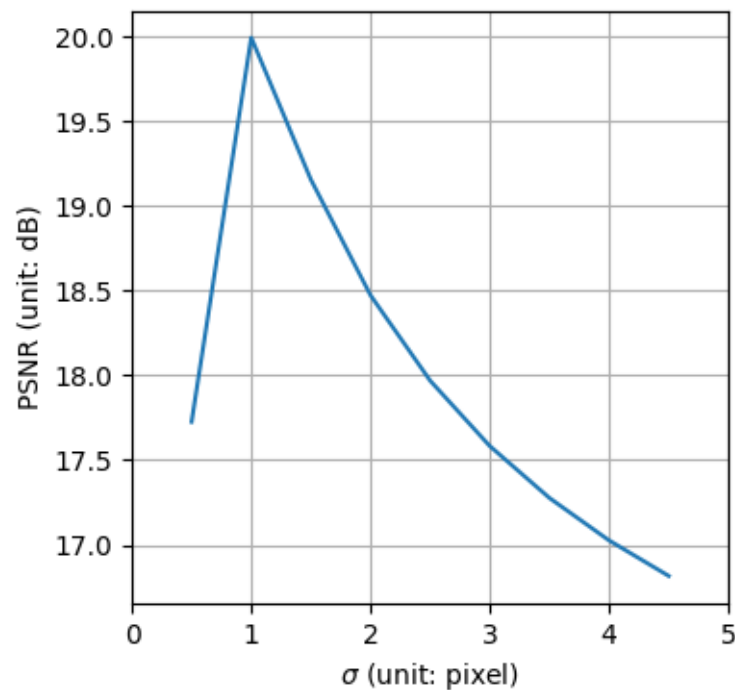
list_psnr = np.array(list_psnr)

# Plot the PSNR metric against sigma (provided)
plt.plot(list_sigma, list_psnr)
plt.xlim([0, 5])

```

```
plt.xlabel('$\sigma$ (unit: pixel)')
plt.ylabel('PSNR (unit: dB)')
plt.grid()
plt.gcf().set_size_inches(4, 4)
```

```
<>:35: SyntaxWarning: invalid escape sequence '\s'
<>:35: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipykernel_135680/1135958519.py:35: SyntaxWarning: invalid escape sequence
'\s'
plt.xlabel('$\sigma$ (unit: pixel)')
```



1.4.6 Q2.6 Implement 3x3 Sobel filters, perform Sobel filtering for the noisy image, and display the gradient magnitude map.

```
[10]: # Construct the Sobel filters
      ### Insert your code ###
      sobel_x = np.array([
          [-1, 0, 1],
          [-2, 0, 2],
          [-1, 0, 1]
      ], dtype=np.float64)
      sobel_y = np.array([
          [-1, -2, -1],
          [0, 0, 0],
```

```

    [ 1, 2, 1]
], dtype=np.float64)

# Print the filters (provided)
print('Sobel_x = {0}'.format(sobel_x))
print('Sobel_y = {0}'.format(sobel_y))

# Sobel filtering for the noisy image
### Insert your code ###
grad_x_noisy = scipy.signal.convolve2d(image_noisy, sobel_x, mode='same',
    ↪boundary='symm')
grad_y_noisy = scipy.signal.convolve2d(image_noisy, sobel_y, mode='same',
    ↪boundary='symm')

# Calculate the gradient magnitude
### Insert your code ###
grad_mag_noisy = np.sqrt(grad_x_noisy**2 + grad_y_noisy**2)

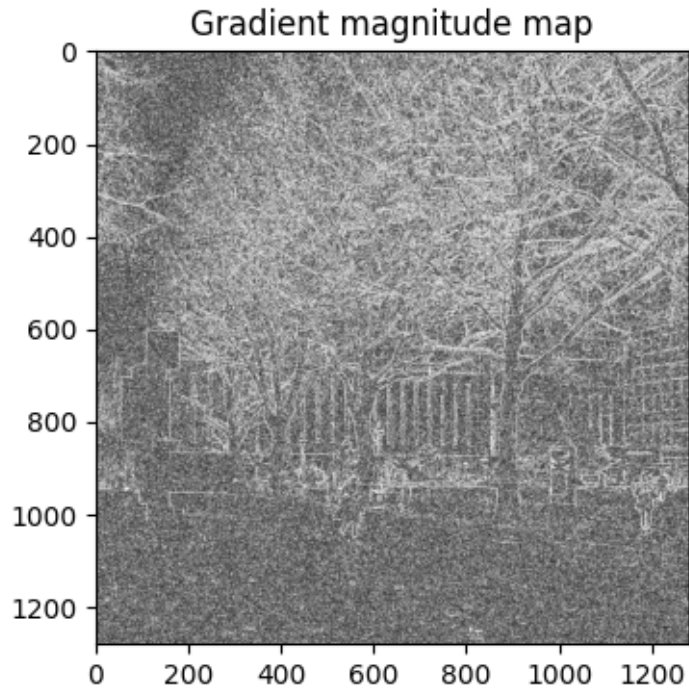
# Display the magnitude map (provided)
plt.imshow(grad_mag_noisy, cmap='gray', vmin=0, vmax=500)
plt.title('Gradient magnitude map')
plt.gcf().set_size_inches(4, 4)

```

```

Sobel_x = [[-1.  0.  1.]
            [-2.  0.  2.]
            [-1.  0.  1.]]
Sobel_y = [[-1. -2. -1.]
            [ 0.  0.  0.]
            [ 1.  2.  1.]]

```



1.4.7 Q2.7 Perform Gaussian smoothing for the noisy image, followed by Sobel filtering and display the gradient magnitude map.

```
[11]: # Parameter for the Gaussian filter (provided)
sigma = 5

# Gaussian smoothing
### Insert your code ###
g = gaussian_filter_1d(sigma)
h_x = g.reshape(1, -1)
h_y = g.reshape(-1, 1)
temp = scipy.signal.convolve2d(
    image_noisy,
    h_x,
    mode='same',
    boundary='symm'
)
image_smoothed_2d = scipy.signal.convolve2d(
    temp,
    h_y,
    mode='same',
    boundary='symm'
)
# Sobel filtering
```

```

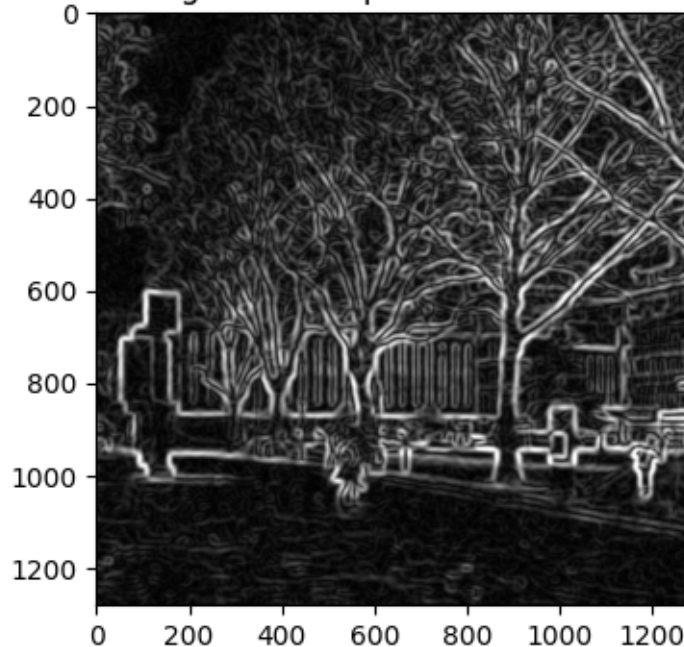
### Insert your code ###
grad_x = scipy.signal.convolve2d(image_smoothed_2d, sobel_x, mode='same',
    ↪boundary='symm')
grad_y = scipy.signal.convolve2d(image_smoothed_2d, sobel_y, mode='same',
    ↪boundary='symm')

# Calculate the gradient magnitude
### Insert your code ###
grad_mag = np.sqrt(grad_x**2 + grad_y**2)

# Display the magnitude map (provided)
plt.imshow(grad_mag, cmap='gray', vmin=0, vmax=100)
plt.title('Gradient magnitude map after Gaussian smoothing')
plt.gcf().set_size_inches(4, 4)

```

Gradient magnitude map after Gaussian smoothing



1.5 Q3. Implement image filters using Pytorch (10 points).

[Pytorch](#) is a machine learning framework that supports filtering and convolution.

The [Conv2D](#) operator takes an input array of dimension $N \times C_1 \times X \times Y$, applies the filter and outputs an array of dimension $N \times C_2 \times X \times Y$. Here, since we only have one image with one colour channel, we will set $N=1$, $C_1=1$ and $C_2=1$. You can read the documentation of [Conv2D](#) for more detail.

```
[12]: # Import libraries (provided)
import torch
```

1.5.1 Q3.1 Expand the dimension of the noisy image into 1x1xXxY and convert it to a Pytorch tensor.

```
[13]: # Expand the dimension of the numpy array
#### Insert your code ####
image_noisy_float = image_noisy.astype(np.float32)
image_noisy_expanded = image_noisy_float[np.newaxis, np.newaxis, :, :]

# Convert to a Pytorch tensor using torch.from_numpy
#### Insert your code ####
tensor_noisy = torch.from_numpy(image_noisy_expanded)
```

1.5.2 Q3.2 Create a Pytorch Conv2D filter, set its kernel to be a 2D Gaussian filter, perform filtering, report computational time and display the result.

```
[15]: # A 2D Gaussian filter when sigma = 5 pixel (provided)
sigma = 5
h = gaussian_filter_2d(sigma)

# Construct the Conv2D filter
#### Insert your code ####
kernel_size = h.shape[0]

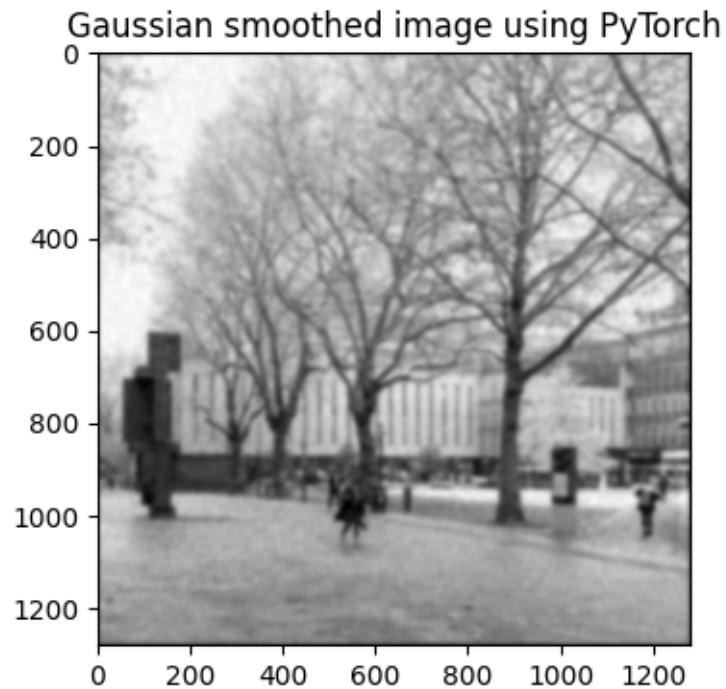
conv = torch.nn.Conv2d(
    in_channels=1,
    out_channels=1,
    kernel_size=kernel_size,
    padding=kernel_size // 2,
    bias=False
)
kernel_tensor = torch.from_numpy(h.astype(np.float32))
kernel_tensor = kernel_tensor.unsqueeze(0).unsqueeze(0)

# Filtering and assess computational time
#### Insert your code ####
with torch.no_grad():
    conv.weight.copy_(kernel_tensor)
conv.eval()
start = time.time()
with torch.no_grad():
    output = conv(tensor_noisy)
torch_time = time.time() - start
print(f"PyTorch Conv2D filtering time = {torch_time:.6f} s")
```

```
image_filtered = output.squeeze().numpy()

# Display the filtering result (provided)
plt.imshow(image_filtered, cmap='gray')
plt.title('Gaussian smoothed image using PyTorch')
plt.gcf().set_size_inches(4, 4)
```

PyTorch Conv2D filtering time = 6.936109 s



[]: